

```
In [1]: import pandas as pd
```

```
In [2]: emp = pd.read_excel(r"C:\Users\suras\OneDrive\NARESH IT\Practice tasks\Rawdata.xlsx")
```

```
In [3]: emp
```

```
Out[3]:
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience#\$	34 years	Mumbai	5^00#0	2+
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1\$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^\$0	10+

```
In [4]: emp.shape # dimentions of the dataFrames
```

```
Out[4]: (6, 6)
```

```
In [5]: len(emp)
```

```
Out[5]: 6
```

```
In [6]: emp.columns
```

```
Out[6]: Index(['Name', 'Domain', 'Age', 'Location', 'Salary', 'Exp'], dtype='object')
```

```
In [7]: len(emp.columns)
```

```
Out[7]: 6
```

```
In [8]: emp.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Name        6 non-null      object
1   Domain      6 non-null      object
2   Age         4 non-null      object
3   Location    4 non-null      object
4   Salary      6 non-null      object
5   Exp         5 non-null      object
dtypes: object(6)
memory usage: 420.0+ bytes
```

```
In [9]: emp
```

Out[9]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience#\$	34 years	Mumbai	5^00#0	2+
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1\$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^\$0	10+

In [10]: emp['Name']

Out[10]: 0 Mike
1 Teddy^
2 Uma#r
3 Jane
4 Uttam*
5 Kim
Name: Name, dtype: object

In [11]: emp['Domain']

Out[11]: 0 Datascience#\$
1 Testing
2 Dataanalyst^^#
3 Ana^^lytics
4 Statistics
5 NLP
Name: Domain, dtype: object

In [12]: emp['Age']

Out[12]: 0 34 years
1 45' yr
2 NaN
3 NaN
4 67-yr
5 55yr
Name: Age, dtype: object

In [13]: emp['Location']

Out[13]: 0 Mumbai
1 Bangalore
2 NaN
3 Hyderbad
4 NaN
5 Delhi
Name: Location, dtype: object

In [14]: emp['salary']

```

-----
KeyError                                Traceback (most recent call last)
File ~\anaconda2\Lib\site-packages\pandas\core\indexes\base.py:3805, in Index.get_loc(self, key)
    3804 try:
-> 3805     return self._engine.get_loc(casted_key)
    3806 except KeyError as err:

File index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas\_libs\hashtable_class_helper.pxi:7081, in pandas._libs.hashtable.PyObjectHashTable.get_item()

File pandas\_libs\hashtable_class_helper.pxi:7089, in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'salary'

The above exception was the direct cause of the following exception:

KeyError                                Traceback (most recent call last)
Cell In[14], line 1
----> 1 emp['salary']

File ~\anaconda2\Lib\site-packages\pandas\core\frame.py:4102, in DataFrame.__getitem__(self, key)
    4100 if self.columns.nlevels > 1:
    4101     return self._getitem_multilevel(key)
-> 4102 indexer = self.columns.get_loc(key)
    4103 if is_integer(indexer):
    4104     indexer = [indexer]

File ~\anaconda2\Lib\site-packages\pandas\core\indexes\base.py:3812, in Index.get_loc(self, key)
    3807 if isinstance(casted_key, slice) or (
    3808     isinstance(casted_key, abc.Iterable)
    3809     and any(isinstance(x, slice) for x in casted_key)
    3810 ):
    3811     raise InvalidIndexError(key)
-> 3812     raise KeyError(key) from err
    3813 except TypeError:
    3814     # If we have a listlike key, _check_indexing_error will raise
    3815     # InvalidIndexError. Otherwise we fall through and re-raise
    3816     # the TypeError.
    3817     self._check_indexing_error(key)

KeyError: 'salary'

```

```
In [15]: emp['Salary']
```

```
Out[15]: 0      5^00#0
          1      10%%000
          2      1$5%000
          3      2000^0
          4      30000-
          5      6000^$0
          Name: Salary, dtype: object
```

```
In [16]: emp['Exp']
```

```
Out[16]: 0      2+
          1      <3
          2      4> yrs
          3      NaN
          4      5+ year
          5      10+
          Name: Exp, dtype: object
```

```
In [17]: emp[['Name', 'Domain']]
```

```
Out[17]:
```

	Name	Domain
0	Mike	Datascience#\$
1	Teddy^	Testing
2	Uma#r	Dataanalyst^^#
3	Jane	Ana^^lytics
4	Uttam*	Statistics
5	Kim	NLP

```
In [18]: emp[['Name', 'Domain', 'Age']]
```

```
Out[18]:
```

	Name	Domain	Age
0	Mike	Datascience#\$	34 years
1	Teddy^	Testing	45' yr
2	Uma#r	Dataanalyst^^#	NaN
3	Jane	Ana^^lytics	NaN
4	Uttam*	Statistics	67-yr
5	Kim	NLP	55yr

```
In [19]: emp[['Name', 'Domain', 'Age', 'Location']]
```

Out[19]:

	Name	Domain	Age	Location
0	Mike	Datascience#\$	34 years	Mumbai
1	Teddy^	Testing	45' yr	Bangalore
2	Uma#r	Dataanalyst^^#	NaN	NaN
3	Jane	Ana^^lytics	NaN	Hyderabad
4	Uttam*	Statistics	67-yr	NaN
5	Kim	NLP	55yr	Delhi

Data Cleansing

In [20]: emp['Name']

Out[20]:

```
0    Mike
1  Teddy^
2   Uma#r
3     Jane
4  Uttam*
5     Kim
Name: Name, dtype: object
```

In [21]: emp['Name'] = emp['Name'].str.replace(r"\W", "", regex = True)

In [22]: emp['Name']

Out[22]:

```
0    Mike
1  Teddy
2   Umar
3     Jane
4  Uttam
5     Kim
Name: Name, dtype: object
```

In [23]: emp['Domain'] = emp['Domain'].str.replace(r"\W", "", regex = True)

In [24]: emp['Domain']

Out[24]:

```
0    Datascience
1      Testing
2    Dataanalyst
3      Analytics
4    Statistics
5           NLP
Name: Domain, dtype: object
```

In [25]: emp['Age'] = emp['Age'].str.replace(r"\W", "", regex=True)

In [26]: emp['Age']

```
Out[26]: 0    34years
         1     45yr
         2      NaN
         3      NaN
         4     67yr
         5     55yr
         Name: Age, dtype: object
```

```
In [27]: emp['Age'] = emp['Age'].str.extract('(\d+)')
```

```
<>:1: SyntaxWarning: invalid escape sequence '\d'
<>:1: SyntaxWarning: invalid escape sequence '\d'
C:\Users\suras\AppData\Local\Temp\ipykernel_27192\2517209319.py:1: SyntaxWarning: in
valid escape sequence '\d'
    emp['Age'] = emp['Age'].str.extract('(\d+)')
```

```
In [28]: emp['Age']
```

```
Out[28]: 0     34
         1     45
         2    NaN
         3    NaN
         4     67
         5     55
         Name: Age, dtype: object
```

```
In [29]: emp['Location'] = emp['Location'].str.replace(r"\W", "", regex=True)
```

```
In [30]: emp['Location']
```

```
Out[30]: 0      Mumbai
         1   Bangalore
         2        NaN
         3    Hyderabad
         4        NaN
         5        Delhi
         Name: Location, dtype: object
```

```
In [31]: emp['Salary'] = emp['Salary'].str.replace(r"\W", "", regex=True)
```

```
In [32]: emp['Salary']
```

```
Out[32]: 0     5000
         1    10000
         2    15000
         3    20000
         4    30000
         5    60000
         Name: Salary, dtype: object
```

```
In [36]: emp['Exp'] = emp['Exp'].str.extract('(\d+)')
```

```
<>:1: SyntaxWarning: invalid escape sequence '\d'
<>:1: SyntaxWarning: invalid escape sequence '\d'
C:\Users\suras\AppData\Local\Temp\ipykernel_27192\3836251810.py:1: SyntaxWarning: in
valid escape sequence '\d'
emp['Exp'] = emp['Exp'].str.extract('(\d+)')
```

In [37]: `emp['Exp']`

Out[37]:

0	2
1	3
2	4
3	NaN
4	5
5	10

Name: Exp, dtype: object

In [38]: `emp`

Out[38]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	NaN	NaN	15000	4
3	Jane	Analytics	NaN	Hyderbad	20000	NaN
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

In [39]: `clean_data = emp.copy()`

In [40]: `clean_data`

Out[40]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	NaN	NaN	15000	4
3	Jane	Analytics	NaN	Hyderbad	20000	NaN
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

Missing value treatment

In [41]: `clean_data.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Name         6 non-null      object
1   Domain        6 non-null      object
2   Age           4 non-null      object
3   Location      4 non-null      object
4   Salary        6 non-null      object
5   Exp           5 non-null      object
dtypes: object(6)
memory usage: 420.0+ bytes
```

```
In [42]: import numpy as np
```

```
In [43]: clean_data.head(2)
```

```
Out[43]:
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3

```
In [44]: clean_data['Age']
```

```
Out[44]: 0      34
1      45
2     NaN
3     NaN
4      67
5      55
Name: Age, dtype: object
```

```
In [47]: clean_data['Age'] = clean_data['Age'].fillna(np.mean(pd.to_numeric(clean_data['Age']
```

```
In [48]: clean_data['Age']
```

```
Out[48]: 0      34
1      45
2     50.25
3     50.25
4      67
5      55
Name: Age, dtype: object
```

```
In [49]: emp
```


Out[49]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	NaN	NaN	15000	4
3	Jane	Analytics	NaN	Hyderbad	20000	NaN
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

In [50]: `clean_data`

Out[50]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	50.25	NaN	15000	4
3	Jane	Analytics	50.25	Hyderbad	20000	NaN
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

In [51]: `clean_data['Exp'] = clean_data['Exp'].fillna(np.mean(pd.to_numeric(clean_data['Exp']`In [52]: `clean_data['Exp']`

Out[52]:

0	2
1	3
2	4
3	4.8
4	5
5	10

Name: Exp, dtype: object

In [53]: `clean_data`

```
Out[53]:
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	50.25	NaN	15000	4
3	Jane	Analytics	50.25	Hyderabad	20000	4.8
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

```
In [55]: clean_data['Location'] = clean_data['Location'].fillna(clean_data['Location'].mode(
```

```
In [56]: clean_data['Location']
```

```
Out[56]: 0      Mumbai
1      Bangalore
2      Hyderabad
3      Hyderabad
4          NaN
5          Delhi
Name: Location, dtype: object
```

```
In [57]: clean_data
```

```
Out[57]:
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	50.25	Hyderabad	15000	4
3	Jane	Analytics	50.25	Hyderabad	20000	4.8
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

```
In [60]: clean_data.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   Name        6 non-null      object
 1   Domain      6 non-null      object
 2   Age         6 non-null      object
 3   Location    5 non-null      object
 4   Salary      6 non-null      object
 5   Exp         6 non-null      object
dtypes: object(6)
memory usage: 420.0+ bytes

```

```
In [61]: # convert object datatype to other using astype function
clean_data['Age'] = clean_data['Age'].astype(int)
```

```
In [62]: clean_data['Salary'] = clean_data['Salary'].astype(int)
```

```
In [63]: clean_data['Exp'] = clean_data['Exp'].astype(int)
```

```
In [65]: clean_data.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   Name        6 non-null      object
 1   Domain      6 non-null      object
 2   Age         6 non-null      int32
 3   Location    5 non-null      object
 4   Salary      6 non-null      int32
 5   Exp         6 non-null      int32
dtypes: int32(3), object(3)
memory usage: 348.0+ bytes

```

```
In [66]: clean_data
```

```
Out[66]:
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	50	Hyderbad	15000	4
3	Jane	Analytics	50	Hyderbad	20000	4
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

```
In [67]: clean_data['Name'] = clean_data['Name'].astype('category')
```

```
In [68]: clean_data['Domain'] = clean_data['Domain'].astype('category')
```

```
In [69]: clean_data['Location'] = clean_data['Location'].astype('category')
```

```
In [70]: clean_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Name        6 non-null     category
1   Domain      6 non-null     category
2   Age         6 non-null     int32
3   Location    5 non-null     category
4   Salary      6 non-null     int32
5   Exp         6 non-null     int32
dtypes: category(3), int32(3)
memory usage: 866.0 bytes
```

```
In [71]: clean_data
```

```
Out[71]:
```

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	50	Hyderbad	15000	4
3	Jane	Analytics	50	Hyderbad	20000	4
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

```
In [72]: clean_data.to_csv('clean_data.csv')
```

```
In [73]: import os
os.getcwd()
```

```
Out[73]: 'C:\\Users\\suras'
```

```
In [74]: clean_data.columns
```

```
Out[74]: Index(['Name', 'Domain', 'Age', 'Location', 'Salary', 'Exp'], dtype='object')
```

```
In [75]: import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [76]: import warnings
warnings.filterwarnings('ignore')
```

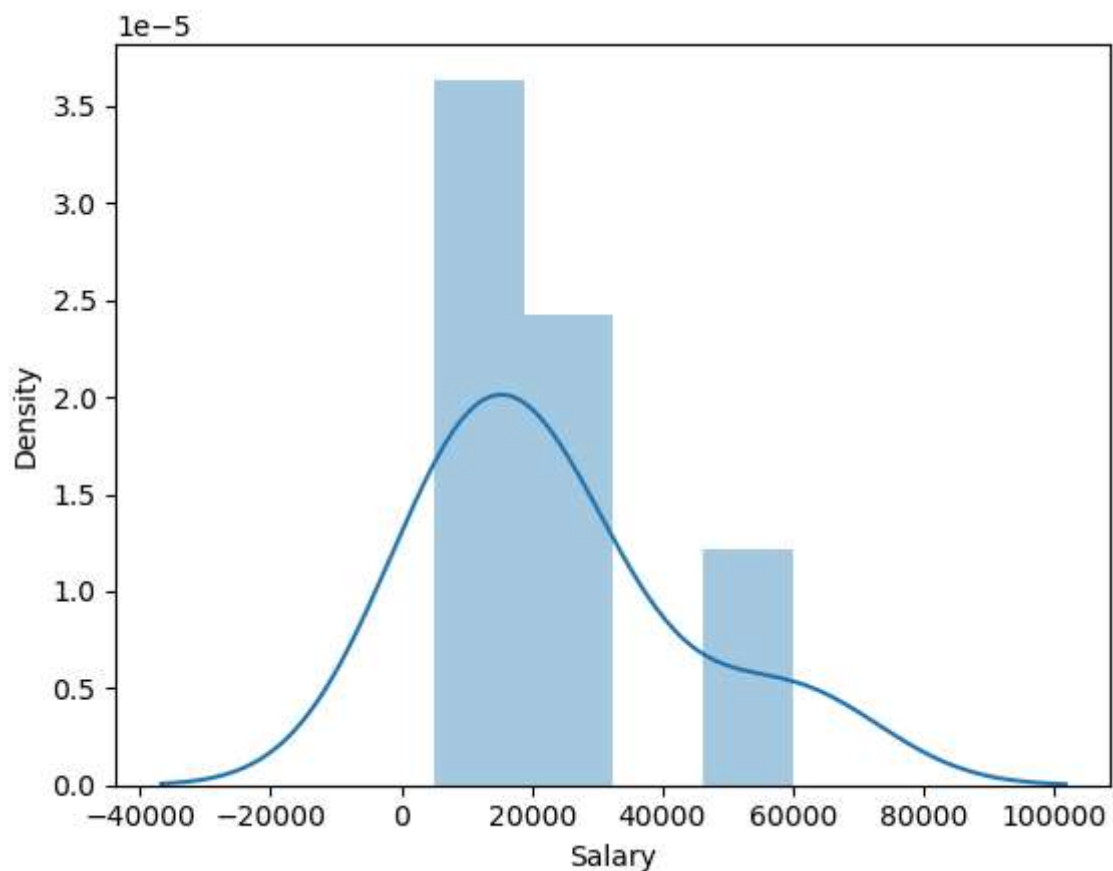
```
In [77]: clean_data
```

Out[77]:

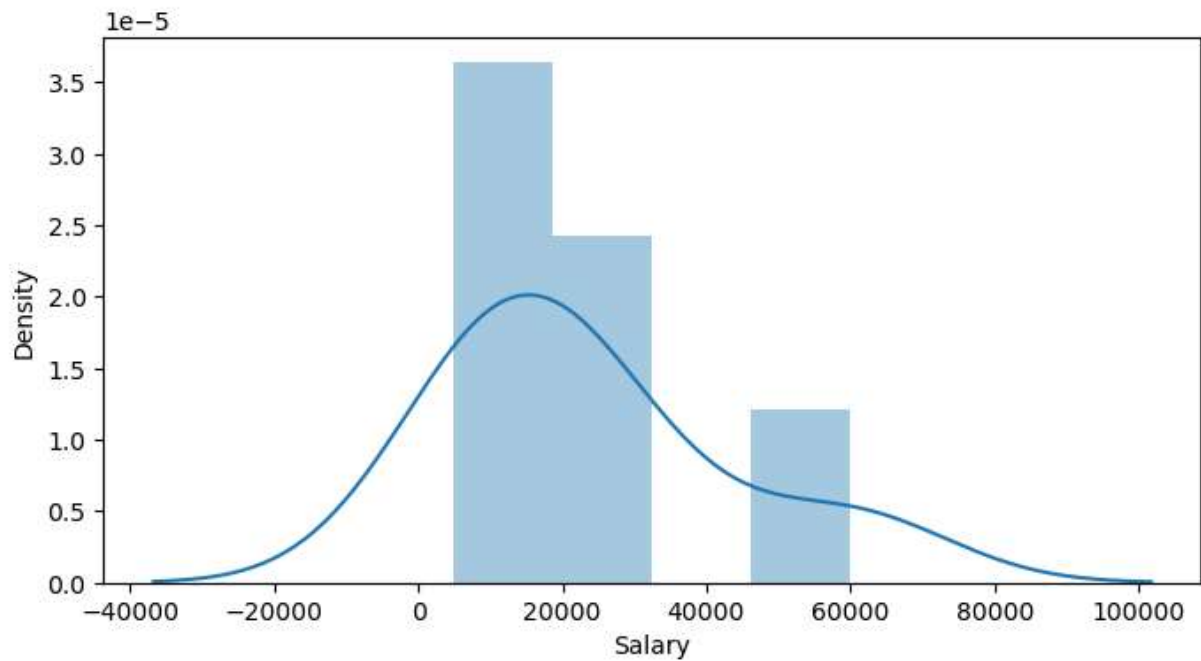
	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	50	Hyderbad	15000	4
3	Jane	Analytics	50	Hyderbad	20000	4
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

In [78]: `clean_data['Salary']`

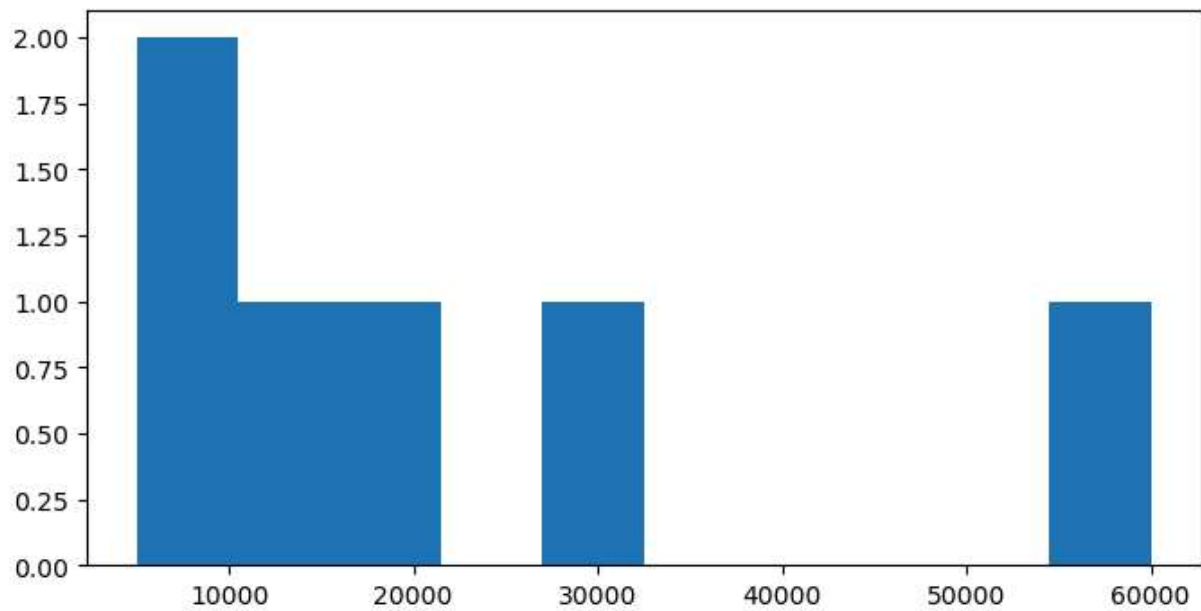
```
Out[78]: 0    5000
1   10000
2   15000
3   20000
4   30000
5   60000
Name: Salary, dtype: int32
```

In [79]: `vis1 = sns.distplot(clean_data['Salary']) # univariate analysis`In [81]: `plt.rcParams['figure.figsize'] = 8,4`

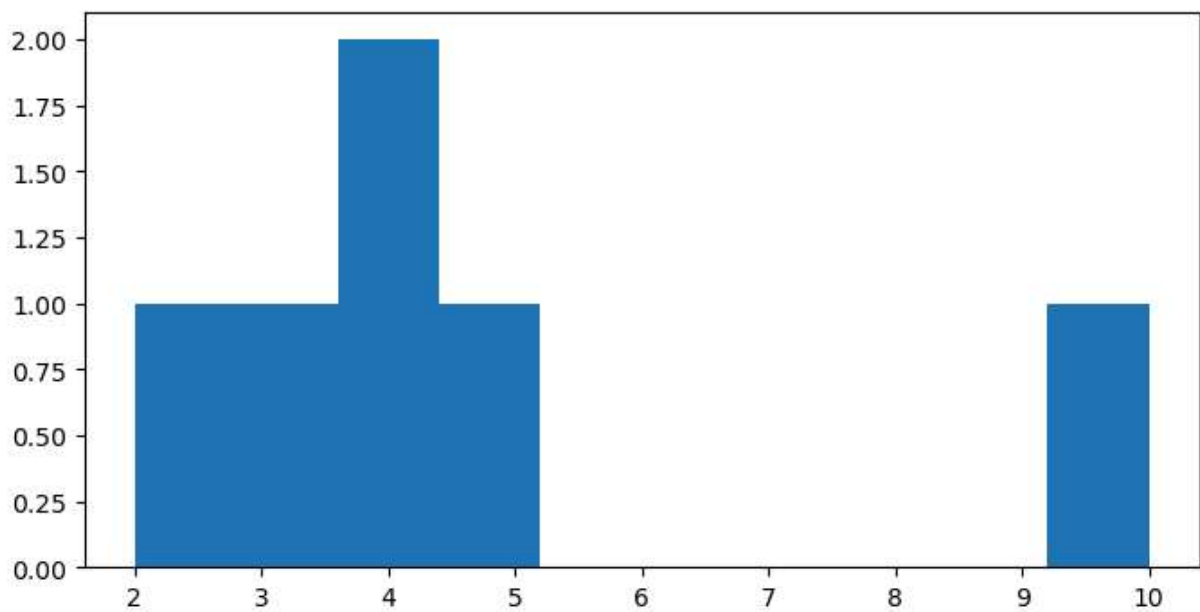
```
In [82]: vist1 = sns.distplot(clean_data['Salary'])
```



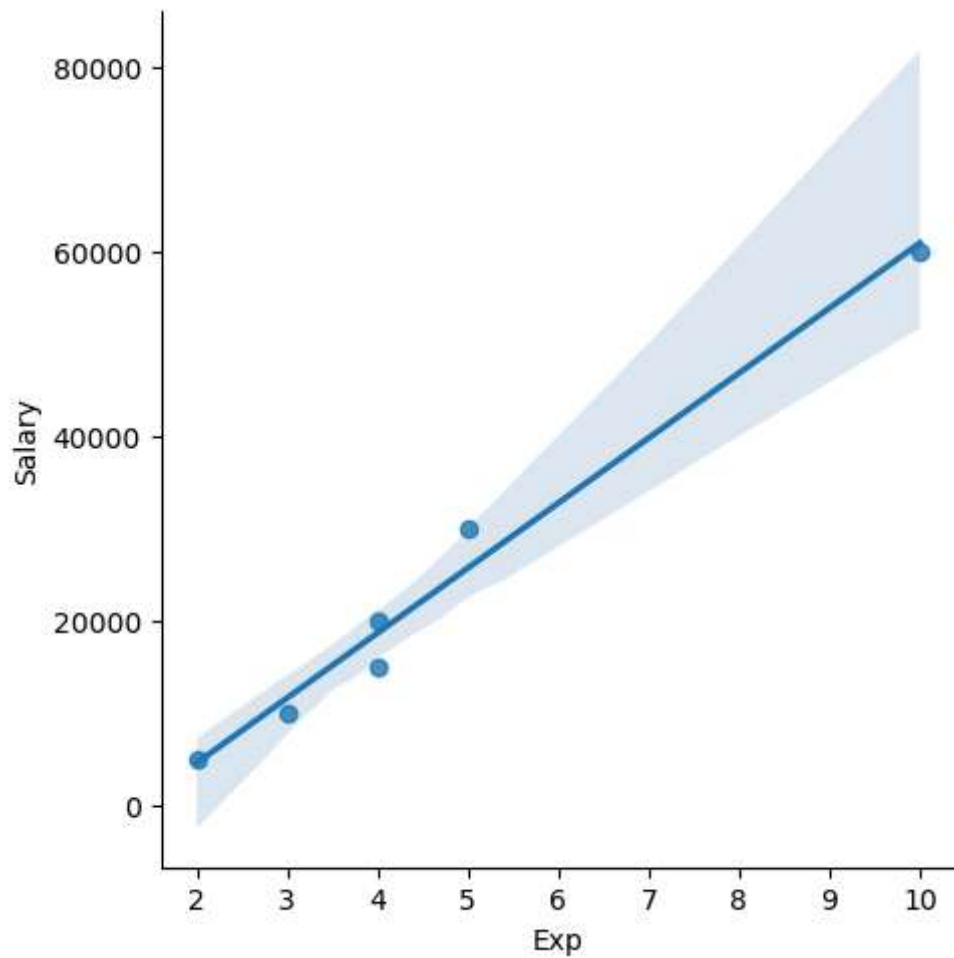
```
In [83]: vis2 = plt.hist(clean_data['Salary'])
```



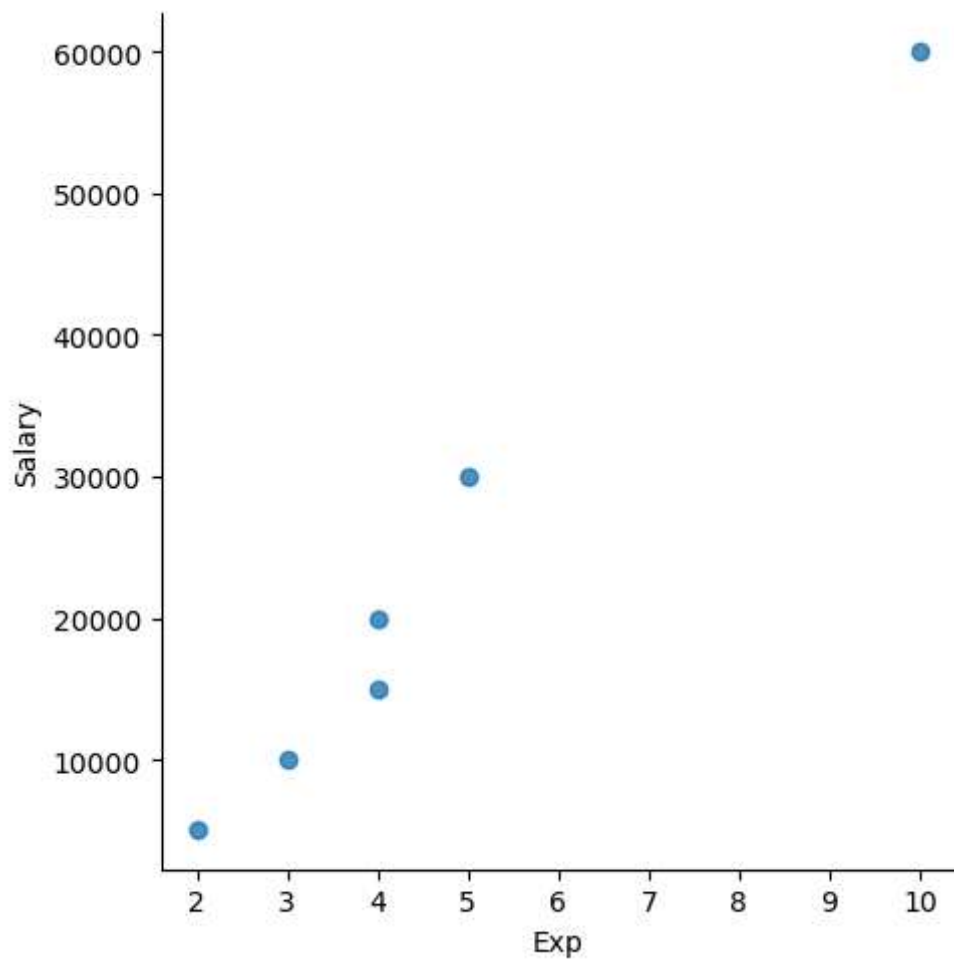
```
In [84]: vis2 = plt.hist(clean_data['Exp'])
```



```
In [86]: vis3 = sns.lmplot(data = clean_data, x = 'Exp', y='Salary')
```



```
In [88]: vis4 = sns.lmplot(data = clean_data, x = 'Exp', y='Salary', fit_reg = False)
```



In [89]: `clean_data`

Out[89]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	50	Hyderbad	15000	4
3	Jane	Analytics	50	Hyderbad	20000	4
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

In [90]: `clean_data[:2]`

Out[90]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3

In [91]: `clean_data[:,]`

Out[91]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	50	Hyderabad	15000	4
3	Jane	Analytics	50	Hyderabad	20000	4
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

In [92]: *# splitting data variables into dependant and independent variable*
`x_iv = clean_data.drop(['Salary'],axis = 1)`

In [93]: `clean_data`

Out[93]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	50	Hyderabad	15000	4
3	Jane	Analytics	50	Hyderabad	20000	4
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

In [95]: `y_dv = clean_data.drop(['Name','Domain','Age','Location','Exp'],axis = 1)`

In [96]: `y_dv`

Out[96]:

	Salary
0	5000
1	10000
2	15000
3	20000
4	30000
5	60000

Imputation

```
In [97]: imputation = pd.get_dummies(clean_data)
```

```
In [98]: imputation
```

Out[98]:

	Age	Salary	Exp	Name_Jane	Name_Kim	Name_Mike	Name_Teddy	Name_Umar	Name_Vincent
0	34	5000	2	False	False	True	False	False	False
1	45	10000	3	False	False	False	True	False	False
2	50	15000	4	False	False	False	False	True	False
3	50	20000	4	True	False	False	False	False	False
4	67	30000	5	False	False	False	False	False	False
5	55	60000	10	False	True	False	False	False	False

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```